# Blatt 06: Cycle Chronicles (ÄK&GW, Mocking)

Carsten Gips (HSBI)

**Unterlagen**

- PDF-Version

---

Forken Sie das "Cycle Chronicles"-Repo und erzeugen Sie sich eine lokale Arbeitskopie von Ihrem Fork.

Bearbeiten Sie die folgenden Teilaufgaben jeweils in einem eigenen Branch. Pushen Sie Ihre Änderungen in Ihren Fork zurück und erstellen Sie dort je einen Pull-Request auf **Ihren** eigenen `master`-Branch.

**Bitte lassen Sie die Pull-Requests bis zur Vorstellung im Praktikum offen.**

Achten Sie darauf, alle Schritte nachvollziehbar in Ihrer Arbeitskopie per Git-Commit festzuhalten. Demonstrieren Sie im Praktikum, wie Sie mit den Pull-Requests arbeiten.

## 1 Analyse: Äquivalenzklassen & Grenzwerte

Die Methode `Shop#accept` dient zur Annahme eines neuen Auftrags eines Kunden.

Neue Aufträge sollen nur unter bestimmten Bedingungen angenommen werden:

- Es darf sich nicht um ein E-Bike handeln.
- Es darf sich nicht um ein Gravel-Bike handeln.
- Der Kunde darf nicht noch andere offene Aufträge beim Shop haben (es kann pro Kunden immer nur maximal einen offenen Auftrag in der Warteschlange geben).
- Es sind aktuell höchstens vier andere offene Aufträge vorhanden (es dürfen zu jeder Zeit maximal fünf offene Aufträge in der Warteschlange sein).

Der Rückgabewert der Methode signalisiert, ob der Auftrag angenommen wurde und in die Warteschlange der offenen Aufträge eingereiht wurde (`true`) oder ob er abgelehnt wurde (`false`).

Aufgaben:

1. Erstellen Sie eine Äquivalenzklassen- und Grenzwert-Analyse.
2. Erstellen Sie aus den ermittelten ÄK und GW konkrete Testfälle. (**Noch keine Implementierung!**)

## 2 Mocking I

Implementieren Sie nun die in der vorigen Aufgabe ermittelten Testfälle für die Methode `Shop#accept` mit Hilfe von JUnit (Version 4 oder 5).

Leider gibt es beim Ausführen vieler Ihrer JUnit-Testmethoden eine `UnsupportedOperationException`-Exception, da die Klasse `Order` bisher nur unvollständig implementiert ist: Es existieren praktisch nur die Methodensignaturen, der Aufruf der Methoden liefert nur eine `UnsupportedOperationException`.

Setzen Sie aktiv Mocking mit Mockito ein, um Ihre JUnit-Tests für `Shop#accept` ausführbar zu machen. Begründen Sie die Anwendung von Mockito.

**Wichtig**: Die zu testende Methode `Shop#accept` soll in der vorliegenden Implementierung im Test genutzt werden, d.h. sie darf nicht "weg-gemockt" werden!

*Tipp*: In der Gradle-Konfiguration der Vorgabe ist bereits JUnit5 und Mockito vorkonfiguriert, d.h. die entsprechenden Abhängigkeiten werden durch Gradle aufgelöst. Wenn Sie die Vorgaben als Gradle-Projekt in Ihrer IDE öffnen, dann steht Ihnen dort auch die JUnit5-Bibliothek automatisch zur Verfügung. Wenn Sie JUnit4 nutzen möchten, müssten Sie bitte die Gradle-Konfiguration entsprechend anpassen. Mit `./gradlew test` können Sie Ihre Testfälle ausführen.

## 3 Mocking II

Die Methoden `Shop#repair` und `Shop#deliver` sind auch noch nicht implementiert. Nutzen Sie geeignetes Mocking, um für diese beiden Methoden Tests in JUnit zu implementieren. Begründen Sie die Anwendung von Mockito.

*Hinweis*: Sie *müssen* hier keine ÄK/GW-Analyse machen, können das aber natürlich gern tun.

## 4 Record-Klassen

Die Klasse `Order` ist zwar bisher nur unvollständig implementiert, aber Sie können bereits deutlich erkennen, dass es zwei Attribute geben muss (welche?).

Bauen Sie die Klasse in eine passende Record-Klasse mit den entsprechenden Attributen um. Sie dürfen die beiden Methoden in `Order` auch geeignet "implementieren" und umbenennen - die Umbenennung muss dann aber auch in den Aufrufen in `Shop` und in Ihren JUnit-Tests passieren!

## 5 Logging

Bauen Sie für den `Shop` ein Logging auf der Basis von `java.util.logging` ein: Jede Änderung an der Auftrags-Warteschlange `pendingOrders` und auch an der Menge der fertigen Aufträge `completedOrders` soll in ein gemeinsames CSV-File geloggt werden. Dabei soll pro Logging-Vorgang eine neue Zeile mit den folgenden Informationen angehängt werden:

- Log-Level,
- Name der den Log-Vorgang auslösenden Methode,
- Name der Klasse, in der die den Log-Vorgang auslösenden Methode angesiedelt ist,
- Log-Meldung, bestehend aus
  - den beiden Details der Order (Fahrradtyp, Name des Kunden), und
  - dem Namen der betroffenen Datenstruktur ("pendingOrders" oder "completedOrders").

Demonstrieren Sie in der Abgabe, wie Sie im Test oder im Hauptprogramm den Logger steuern können, beispielsweise Änderung der Log-Level oder Abschalten des Loggings.

*Tipp*: Der Aufruf von `Shop#repair` ändert sowohl `pendingOrders` als auch `completedOrders` - hier müssen also beim Logging zwei neue Zeilen im Logfile angelegt werden.

**Hinweis**: Da in den Klassen der Vorgabe die meisten Methoden nicht implementiert sind, müssen Sie dies für diese Aufgabe selbst flink erledigen. Es handelt sich um die vier Methoden `Shop#repair` und `Shop#deliver` sowie `Order#getBicycleType` und `Order#getBicycleType`.[1]

[1] Die Methoden `Order#getBicycleType` und `Order#getBicycleType` haben Sie sogar schon bei der Umsetzung der Record-Klassen-Aufgaben "implementiert" :-)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 8aa3159 (tooling: do not generate beamer slides for readme's and homework, 2025-05-11)